



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



EXPLAIN CLONING IN JAVA WITH EXAMPLE

CREATING A COPY OF AN EXISTING OBJECT



- Cloning in Java means creating a copy of an existing object. 📄✨
- It is one of the ways to create a new object in Java. 🔄

For example:

- 📝 Imagine you want to create a new resume, but you don't know how to start.
- 🏠 Your friend has a very good resume, so you decide to take a copy of it and edit it.
- ✅ This means you have cloned your friend's resume to create your own.

In Detail: Cloning in Java 🖥️☕

- 🆕 Cloning in Java is one of the ways to create a new object.
- 📄 When we create a new object using the cloning technique, we get an exact copy of the existing object with the same state and behaviour.
- 🛠️ To achieve this, our class must implement the Cloneable interface and override the clone() method.
- ⚠️ If not, we will encounter a CloneNotSupportedException.



⚠️ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



TYPES OF CLONING IN JAVA



1. Shallow Copy 🔄

- Creates a copy of an object using reference.
- Copies only the reference of the object to the new object; the objects themselves are not duplicated.
- Changes to the original object are reflected in the new object, and vice versa.
- Performed by default when using the Object class's clone() method.

2. Deep Copy 🔄

- Creates a copy of an object using constructor initialization.
- New objects are created for all reference types.
- Changes in the original object do not affect the new object, and vice versa.
- Requires manual implementation by recursively copying all fields of the original object.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SHALLOW COPY VS. DEEP COPY

Shallow Copy vs. Deep Copy ⚖️

Shallow Copy 🌀	Deep Copy 🔄
▪ Creates a copy of an object using reference. ▪ 🚀 Fast as no new memory is allocated. ▪ 🔧 Changes in one entity are reflected in the other entity. ▪ 📦 Default behaviour of the clone() method. ▪ 💰 Less expensive.	▪ Creates a copy of an object using constructor initialization. ▪ 🐢 Slow as new memory is allocated. ▪ 🗝 Changes in one entity do not reflect in the other entity. ▪ Requires overriding the clone() method to implement deep copy. ▪ 💰 Highly expensive.

Case Study in Cloning 📖

Are Shallow Copy and Deep Copy Related to Cloning?

- ✅ Yes, they are!
- Cloning in Java creates a new object as an exact copy of an existing object, either through shallow copy or deep copy.

How?

Shallow Copy:

- By default, the clone() method of the Object class performs a shallow copy.
- Reference fields in the cloned object point to the same objects as in the original object.

Deep Copy:

- To create a deep copy using clone(), override the method and recursively copy all fields, ensuring complete independence from the original object.



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



WHEN TO USE SHALLOW COPY VS. DEEP COPY 🔍

When to Use Shallow Copy vs. Deep Copy 🔍

When to Use Shallow Copy? 🌀

- When you want a new object to share data with the original object while minimizing memory usage.
- Faster and simpler but changes in one object affect the other.
- Example: Temporary data sharing.

When to Use Deep Copy? 🔄

- When you need a new object that is completely independent of the original.
- Changes in one object do not affect the other.
- Example: When the original and copied objects require separate lifecycles.

Summary 📝

- Use deep copy when you need the cloned object to be independent of the original.
- Use shallow copy when sharing data is acceptable, and memory usage needs to be minimized.



⚠️ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding of how memory architecture works.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com